回答范式：

结构要求：几句话结论(必要) + 若干重点(可选) + 一个例子(可选)

语境要求：模拟面试者在面试场景回答问题的语境，以此给出贴切问题的正确答案，重点突出，核心明确。

字数要求：200字以内，具体字数视问题复杂程度而定。

例外：项目自主问答问题不受上述字数限制，需要交代项目背景、个人职责、难点原因、具体方案和实际成果。

Agent面试题

1.什么是AI Agent?

结论：AI Agent 是围绕任务自主决策、执行动作，并根据反馈调整下一步动作的智能体。

重点：在实际应用中，由模型负责决策、工具负责执行，形成“感知—思考—行动”的循环。

2.什么是Harness?

结论：Agent Harness 是围绕大模型的一套运行支撑，核心是组织模型和工具执行的循环。

重点：包含上下文工程、记忆机制、提示词工程、中间件等实现。

3.Agent 的基本架构由哪些核心组件构成?

结论：Agent 的基本架构有四个核心组件：大模型、工具、记忆、规划模块。

重点：大模型是Agent的核心大脑；工具用于执行操作；记忆分为长期和短期，短期记忆保存上下文，长期记忆按需持久化经验；规划模块负责任务拆解和决策，常见设计范式包括**ReAct**和**Plan-and-Execute**。

4. Workflow，Agent，Tools 这三个的概念和区别介绍一下？

结论：Tools 是用代码封装好的一系列操作，用于让Agent调用，执行操作；

Agent 是围绕任务自主决策、执行动作，并根据反馈调整下一步动作的智能体。

Workflow 按开发者定义的规则编排流程。

重点：三者可以组合：Agent和工具可以作为Workflow的一个节点被编排。

5. 了解哪些 Agent 设计范式？

结论：常见设计范式有 ReAct、Plan-and-Execute、Reflection。

重点：ReAct模式将Agent的执行拆成了感知、思考、行动3个步骤的循环，大模型决定要不要调用工具，如果要调用工具的话，就调用工具之后进入下一轮循环，否则就退出循环返回结果。

Plan-and-Execute是先规划后执行，先生成一个完整的步骤列表，再逐步执行。执行过程中可以根据每一步的执行结果，决定要不要调整后续步骤，即动态重规划。

Reflection是在完成任务之后，让另一个大模型来进行评估结果是否符合预期，如果不符合预期，则让Agent带着评估结果进行重试。代价是会延迟和token消耗都会增加。

如何选型：

任务不复杂，直接用 **ReAct**，够用就好；复杂任务容易跑偏的，使用**Plan-and-Execute**；输出要求高的话，在上面叠加Reflection。

三者可以结合使用，用 Plan-and-Execute 做整体规划，每个步骤内部用 ReAct 来执行，最后再加上 Reflection 做质量把关。

6. CoT是什么？ToT呢？

结论：CoT，全称 Chain of Thought（思维链）

重点：CoT 有两种触发方式，第一种叫 Zero-shot CoT，直接在 prompt 末尾加上「让我们一步步思考」这句话就行了；

第二种叫 Few-shot CoT，你需要在 prompt 里给几个带有完整推理过程的例子，让 LLM 照着这个格式来模仿。

扩展：ToT 的全称是 Tree of Thoughts（思维树），在每次推理的时候，每一步给出多个不同的方向，然后使用另一个大模型评估出，选择最好的方向进行深入。

7. Agent 记忆压缩通常有哪些方法？

结论：常见记忆压缩方法有滑动窗口、摘要压缩、重要性过滤等。

重点：在实际生产中，最常用的做法是保留最近几轮的对话，历史对话则使用摘要算法进行压缩。

8. 在工程实践中，为什么有时候选择「手搓」Agent，而不是直接用成熟框架？

结论：使用成熟框架，如Langchian上手快，不用重复造轮子，缺点是相对不够透明可控。手搓Agent可控性强，但是需要投入更多的人力进行开发维护。

重点：两者如何抉择具体要根据部门实际上的项目需求，人力成本等综合考虑；不能主观评判好坏。

RAG面试题

1. 什么是 RAG？详细描述一个完整 RAG 系统的详细工作流程？

结论：RAG 是检索增强生成：从知识库中检索数据，让LLM结合检索到的知识进行回答。

重点：RAG主要分成两套阶段：离线阶段是文档加载、文档切割、向量化Embedding、存入向量数据库；

在线阶段是查询预处理、向量检索、Rerank重排、组装上下文、LLM生成答案。

2. 大模型的 RAG 主要用来解决什么问题？

结论：RAG 主要解决的是大模型的知识时效性问题和私有知识覆盖的问题。

重点：知识时效性是指LLM 训练完知识就固定了，训练截止日期之后的事情他并不知道；私有知识覆盖是指大模型没有公司内部文档，项目专业知识等。

3. 微调和 RAG 各自的优劣势是什么？

结论：微调是用自己的数据继续训练大模型，可以用于定制模型的行为风格；RAG是让大模型结合知识库中的知识给出正确答案。

重点：微调的缺点是训练成本高，需要标注数据，租用GPU，跑训练，并且更新慢。RAG缺点是多了检索延迟，且受限于检索质量上限。

4. 详细说说文档切割（Chunking）策略？

结论：文档的切割策略有固定大小切割；递归字符分块（先按段落切割，再按句子切割，再按标点切割，直到满足chunk size）；还有父子切割；语义边界切割；切割后让大模型补齐背景上下文等。

重点：为了解决语义被切割掉的问题，通常还会加一个10%~20%的重叠overlap。最终的目的都是为了提升每个Chunk的质量。

5. 在 RAG 中 Embedding 究竟是什么？如何选择和评估一个 Embedding 模型？

结论：Embedding 是指将文本映射成向量，语义相关内容在向量空间更接近。

重点：实际项目中，需要根据模型排行榜、擅长语言领域、数据合规等来综合考虑。我们项目中需要综合考虑本地化部署和中文能力强大，于是选择的是Qwen3-Embedding。

6. 什么是向量数据库？有没有做过向量数据库的对比选型？

结论：向量数据库是专门用来存储和检索高维向量的数据库，核心能力是近似最近邻搜索，也叫 ANN，能在百万甚至亿级的向量里快速找出最相似的几条。

重点：我通常按照数据规模来选择，中小规模场景，如我自己的开源的Agent学习项目，使用的是Qdrant；而我们部门的Agent，底层使用的向量数据库则是Milvus。

7. 讲讲你用的向量数据库？数据量级是多大？性能如何？遇到过性能瓶颈吗？

结论：我们生产环境用的是Mivus，数据量级在百万条向量左右，每条是1024维，用HNSW索引，单次查询的延迟在20到50毫秒。

选Milvus主要是因为它支持分布式部署和读写分离，适合数据量大、并发高的场景。

重点：我遇到过两个比较典型的瓶颈。第一个是内存压力，百万级的1024维向量光原始数据就要好几个GB，后来我们开启了标量量化SQ8，把foat32压成int8，内存直接降到原来的四分之一。

第二个是大批量写入的时候会触发后台Segment合并，影响查询延迟，我们的解法是把批量写入改到业务低峰期，分批小批次写入。

8. 你使用 RAG 给大模型一个输入，系统是怎样的工作流程？

结论：系统会经过：查询预处理、向量检索、Rerank重排、Prompt拼装、大模型生成几个阶段。

重点：预处理的时候可以使用HYDE，让大模型假设一个答案，带着假设的答案去向量搜索。

检索阶段可以使用向量检索+BM25关键字混合检索，召回结果使用RRF算法合并，最终召回结果比单路覆盖面更广、质量更高。

Rerank重排阶段使用Rerank模型精排，进一步过滤得到高质量片段。

大模型生成阶段通常要求生成结果中标注引用来源，方便用户溯源对照，提升可信度。

9. 如何润色用户的 Query（Query Rewrite）？目的是什么？

结论：Query Rewrite 是将用户问题变为更利于检索的表达，同时保留原始意图。

重点：一种是直接改写（如：去掉口语化表达、补齐缩写和指代）；另一种是使用HYDE带着假设的答案去进行向量搜索。

10. RAG 检索优化策略有哪些？

结论：我通常在文档切割阶段、查询预处理阶段、检索阶段、重排序阶段加入合适的优化测试。

重点：在文档切割阶段，使用合适的分块和索引策略，如使用语义切割，父子切割等方式优化；

查询预处理阶段，使用HYDE带着假设的答案去向量库搜索；

检索阶段可以使用向量检索+BM25关键字混合检索，召回结果使用RRF算法合并，最终召回结果比单路覆盖面更广、质量更高。

重排序阶段使用Rerank模型精排，进一步过滤得到高质量片段；

11. 了解哪些更复杂的 RAG 范式？

结论：在我的了解里，RAG 的发展经历了三代演进:Naive RAG 是基础的检索加生成;

Advanced RAG 在检索前后都加了优化，比如Query改写、Rerank、混合检索这些;

Modular RAG 是把各个环节做成模块可以任意组合。

重点：我还知道GraphRAG：将知识库沉淀为知识图谱，擅长解决知识之间关联性强、需要全局视角的场景；

Agentic RAG 把 RAG 嵌入 Agent 循环，让Agent自主决定要不要使用知识库的功能，在实际项目中，我们把RAG查询封装成了一个Tool，让大模型自主选择是否使用。

12. 如何规避 RAG 系统中大模型的幻觉？

结论：RAG 幻觉主要有两类。

第一类是检索没有召回到相关内容，LLM 没有可用的上下文，靠自身知识在编造答案。

第二类是检索内容召回了但 LLM 没有严格遵循，在文档内容的基础上加了自己的推断。

重点：第一是 Promp强约束，明确告知 LLM 只能根据提供的资料回答，资料里没有的就说不知道；

第二是生成后引用核查，每个关键的声明都要在 chunk 里找到来源依据；

第三是结构化输出强制溯源，让 LLM 输出 JSON，每条结论必须附上来源编号。

13. 怎么量化你的 RAG 效果？

结论：首先是检索层评估，我们准备一批问题+对应正确 chunk ID的测评数据，然后评估其Hit@K和MRR是否达到目标；未达到目标则进行分析优化。

其次是生成层评估，可以使用RAGAs框架来进行评估。

最后我们还需要关注线上指标：如在线追问率、转人工率等。

形成离线评测->优化上线->线上观测->离线复现->优化上线的完整闭环。

14. RAG 知识库如何实现动态与持续更新？

结论：我们的通用方案是给每个文档算一个内容 hash，通过轮询或者监听数据源变更，检测到文档新增、修改、删除的时候，先清掉旧的向量，再重新切割入库。

15. 在实际落地中，你觉得 RAG 最难的地方是哪里？

结论：我认为最难的地方在于保证检索质量。我们通过拆分不同的阶段

重点：在文档切割阶段，使用合适的分块和索引策略，如使用语义切割，父子切割等方式优化；

查询预处理阶段，使用HYDE带着假设的答案去向量库搜索；

检索阶段可以使用向量检索+BM25关键字混合检索，召回结果使用RRF算法合并，最终召回结果比单路覆盖面更广、质量更高。

重排序阶段使用Rerank模型精排，进一步过滤得到高质量片段；

最后在评测阶段根据测评结果不断优化。

16. 不同来源的文档发生知识冲突时，RAG 应该信谁？

结论： 我的做法是在知识入库的时候适用范围、来源权威度、版本状态、生效区间等元数据；

当检索的时候发生冲突时，按预设规则裁决：先看适用范围，再看来源权威度，版本状态和生效区间；

优先选择权威度高的文档，如果最终还是无法裁决，则诚实的返回给用户文档冲突情况，列出引用来源，让用户自行判断。

LLM工具调用面试题

1. 什么是 Function Calling？原理是什么？

结论：Function Calling指的是：开发者用 JSON schema 把工具描述好传给模型，模型判断需要调工具的时候输出一段结构化的 tool_calls JSON，指明需要调用的工具和入参，代码再去真正执行，把结果塞回对话，模型再根据执行结果生成最终答案。

重点：核心的设计是，模型全程只做决策，执行的事情一律由宿主代码完成。

2. LLM 是如何学会调用外部工具的？

结论：训练层面，在**SFT**监督微调阶段，给模型喂大量「工具调用示范对话」，让他学会工具调用流程；

在RLHF阶段，训练大模型知道什么时候该调用工具。

在运行层面：开发者用 JSON schema 把工具描述好传给模型，模型判断需要调工具的时候输出一段结构化的 tool_calls JSON，指明需要调用的工具和入参，代码再去真正执行，把结果塞回对话，模型再根据执行结果生成最终答案。

3. 什么是 MCP（模型上下文协议）？讲讲它的核心内容？

结论： MCP 是一套连接 AI 应用与外部工具开放协议。它的优点是可以让工具一次封装，到处复用。

重点： MCP Server封装好一套工具，MCP Client可以和Server进行连接，调用这些工具。

MCP的消息格式是JSON-RCP2.0；

传输协议有Stdio和Streamable HTTP，本地启动的MCP Server使用Stdio，远端的MCP Server则使用Streamable HTTP。

MCP最常用的是暴露Tool能力，除此之外，还有Resources和Prompts模板。

例子：在我的最常用的codex工具中，我们可以在config.toml文件中，按照MCP的格式注册MCP，Codex就有了该MCP提供的所有能力。在一次大模型调用中，codex会调用MCP的tool/list接口，获取到工具定义，将其传递给大模型，大模型返回Tool call之后，codex再调用对应的MCP提供的工具来执行。

4. MCP 和 Function Calling 有什么区别？

结论：Function Calling 解决的是大模型调用工具的问题；MCP是一种工具生态协议，让工具可以一次封装，到处复用，MCP底层的工具调用，还是基于Function Calling的。

5. Skill 是什么？

结论：Agent Skill 是把「指令、脚本、模板」一体化打包成可复用技能包，Agent可以自动发现，按需加载使用技能。

重点：Skill本质上是一个文件夹，里面要一个必不可少的Skill.md文件，还有可选的references、scripts、assets文件夹，分别存放参考文档、脚本、模板文件。

Skill使用渐进式披露的机制，每次只给大模型提供Skill.md中的name和description，当大模型需要的时候，再加载完整的Skill.md，

Skill.md中提到了脚本、参考文档、目标文件，再按需去读取。

例子：现在github上已经有了成千上万的Skill，当我们需要的时候，直接让agent帮忙去github上挑选合适的即可；实在没有合适的，我们也能使用Skill-creator技能自己开发自己的Skill，

例如我在职期间，为了提高测试效率，自己skill-creator开发出了一个基于playwright MCP的端到端测试Skill，用于自动打开浏览器执行测试用例。

6. MCP 和 Agent Skill 的区别是什么？

结论：MCP 是一种工具生态协议，让工具可以一次封装，到处复用；Skill 是把「指令、脚本、模板」一体化打包成可复用技能包。

重点：二者可以互补：Skill 可以指导如何使用 MCP 工具。将MCP工具的调用封装为Skill中的一个流程。

7. 什么是 A2A 协议？

结论：A2A 是 Agent 之间发现能力、委托任务开放协议；

重点：A2A 通过 Agent Card 描述能力，使用TASK状态机来进行互相协作。

8. 说说 WebSocket 和 SSE 通信的区别及局限性？

结论：SSE 是服务端向客户端推送文本事件；WebSocket 是双方都能发送消息的双向连接。

重点： SSE本质上还是使用HTTP协议，在请求头里声明Accept: text/event-stream，表示想要

接收流式数据，服务端收到之后，不关闭连接，不停的往里写数据。

而WebSocket则是独立于HTTP协议之外的一个新的协议。

9. 有没有用过大模型的网关框架？网关层解决了什么问题？

结论：我们公司就有自建大模型网关，此外我还用过LiteLLM，Sub2api这些开源的大模型网关。

重点：网关层解决的问题有3个：第一是多模型统一接口，使用者只需要使用网关的URL和APIKEY，就能通过切换模型的方式，使用网关对接的所有模型；第二个是限流和配额，可以给不同用户设置token预算，防止某个用户把token用光；第三个是成本追踪和可观测性，能够直观的看到某个用户的token使用情况。

大模型工程面试题

1. 什么是大语言模型？

结论：大语言模型是用海量语料预训练、参数百亿到千亿规模、自回归生成文本的统一模型。

2. 讲讲 Transformer 架构基本原理？Encoder 和 Decoder 是什么？

结论：Transformer 最核心是 Self-Attention，让每个 token 都能直接和序列里任意其他位置建立联系，一次性并行计算；为了专注于【预测下一个Token】这个训练目标，采取了Decoder-only 架构。

3. 大模型是怎么训练出来的？

结论：大模型训练通常分为预训练、微调和对齐三个阶段。

预训练阶段使用海量数据让大模型学会【预测下一个词】；

微调让大模型学会根据指令回答问题；

对齐阶段通常使用RLHF（强化学习人类反馈）让模型的回答更符合人类的价值观和偏好。

4. 什么是 Scaling Law？大模型的「涌现能力」是怎么回事？

结论：Scaling Law 描述模型参数规模、训练数据和训练算力与训练损失之间的经验关系，用于配置训练资源。

重点："涌现"指某些大模型能力随规模表现出明显跃升，某些能力会从完成不能直接变成能做，就像是量变引起质变。

5. 大模型微调的方案有哪些？

结论：按更新参数范围，可选全量微调、LoRA 等参数高效微调，以及量化底座上的 QLoRA。

6. 大模型的参数：温度值、Top-P、Top-K 分别是什么？各个场景下的最佳设置是什么？

结论：temperature 控制输出的随机性，越低越稳定可复现，越高越发散有创意；Top-K 是固定从概率最高的 K 个词里选；Top-P 是从累积概率达到 P 的候选词里采样，比 Top-K 更灵活自适应；

重点：我们通常保持Top-P和Top-K为默认值，需要精确问答的时候调低温度，面对创意性问题则调高温度。

7. 大模型为什么会出现幻觉？怎么缓解？

结论：大模型出现幻觉的原因可能有：训练数据有误，大模型引用了错误的信息进行回答；用户询问了大模型不存在的知识，导致大模型编造知识。

重点：针对大模型知识不全，可以使用RAG的方式来优化；幻觉不可能被消除，只能降低发生概率。

8. 大模型部署有哪些主流方案？实际项目里怎么选？

结论：我个人电脑使用Ollama部署过小模型玩过；生产级部署方案了解过vllm和SGlang。

9. 对比使用过哪些主流大模型？你们项目中最终选用了哪个模型？为什么？

结论：主流大模型我想分成三大类来回答，第一类是claude、GPT系列等海外能力标杆，我们通常用于开发任务，但是考虑到安全合规问题，我们不将其用于生产Agent项目接入的大模型；

第二类是国产企业级模型，如GLM、Deepseek、千问系列等，他们都提供了企业级套餐或者token，相对国外大模型在数据合规上会好一点。

第三类是自部署的开源大模型，我们公司租用服务器部署了千问大模型4版本，对于安全级别要求最高的任务使用自部署的大模型。

公司内部搭建的大模型网关，里面有这3大类的模型供我们按需选择。

10. 为什么长上下文会出现 **Lost in the Middle**？上下文窗口越大越好吗？

结论：Lost in the Middle 指长上下文中间的关键信息可能比首尾信息更难被模型有效利用。

重点：上下文窗口并不是越大越好，上下文大了之后，大模型容易出现Lost in the Middle和注意力稀释的现象。

LangChain面试题

1. 你了解过哪些 AI Agent 开发框架？

结论：我熟悉 LangChain、LangGraph，了解过LlamaIndex。

重点：Langchain提供模型、工具、记忆、消息、提示词、中间件等通用抽象，适合快速构建AI应用。

LangGraph偏向于底层的状态管理与流程编排，将流程抽象为节点和边，使用图结构显式控制Agent的执行路径；Langchain的Create Agent底层就是基于Langgraph构建的图，因此二者是上下层关系。

LlamaIndex的优势集中在文档解析，向量索引和检索，适合用于RAG场景。

2. 如何理解 LangChain 中的 Chain？

结论： Chain 是一种应用编排思路：把 Prompt、模型、检索器、输出解析器和自定义逻辑等步骤，按照明确的数据流连接起来，让上一步的输出成为下一步的输入，最终形成一个可以整体执行的流程。它的核心设计理念是「组合」，每一个步骤，都是一个Runable，用LCEL将每个步骤组合起来。

3. 在 LangChain 中，如何为 Agent 注册工具？

结论：通常我们使用@tool定义工具名称、描述和入参；再传入 create_agent 的 tools中。

项目具体问题

1. 请你介绍一下 AI Agent 的记忆机制，并说说你的项目中是如何设计记忆模块的？

结论：Agent 记忆通常分为两种，一种是短期记忆，用于在多轮对话中记住上下文；一种是长期记忆，用于跨会话积累知识。

重点：在我的项目中，短期记忆围绕 `thread_id` 管理，核心是内容是继承 LangChain `AgentState` 的 `ThreadState`。除了对话消息之外，还额外设计了当前任务目标、执行计划等字段；ThreadState通过Checkpointer保存，可以通过配置选择使用内存、SQLite或者PostgreSQL；主要流程是在Agent执行完一个图节点之后更新ThreadState，通过Checkpointer保存ThreadState和图执行信息；使用的时候根据thread_id查询历史ThreadState，合并本轮用户输入；

长期记忆使用user_id为用户隔离边界，保存用户画像，用户偏好，具体事实等。我们开发了两个中间件，一个是在Agent执行前，根据用户id查询用户的长期记忆并注入到上下文中；一个是在Agent执行后，筛选用户输入和助手的最终回复，异步调用大模型提取和更新记忆，提取的时候，会检查信息是否属于用户，是否具有长期价值等。

2. 什么是 Multi-Agent？你们项目中的muti-Agent是如何设计的？

结论：Multi-Agent 是多个具有独立职责和上下文的Agent，通过协作共同完成任务。

重点：优点是每个Agent具有独立的上下文，不容易被信息撑爆；且每个Agent可以有自己的专用系统提示词，处理专门的任务。

模式：多Agent从简单到复杂可以分为：顺序流水线模式、路由模式、并行汇聚模式、监督者模式等。

例子：我们项目采用以主 Agent 为监督者的 Multi-Agent 架构。主 Agent 按需将任务委派给具有独立上下文、专用提示词和工具配置的子 Agent；

独立子任务可以并行执行，结果由主 Agent统一检查、汇总并交付。

在具体设计中，将子Agent调用封装成了Task Tool；通过工具的形式将子Agent的能力暴露给主Agent。

3. 你们Agent项目的上下文工程怎么设计？

结论：我们项目中，上下文工程设计的核心是：在有限的上下文预算内，让 Agent 持续拿到相关信息，保证长任务执行的连续性和准确性。

重点：我们主要围绕这3个重点来设计：

第一是分层管理：系统提示词、历史对话、任务状态和长期记忆分别管理；通过中间件在模型调用前组装这些信息，每类信息有明确的来源、保存方式。

第二是按需加载，控制上下文噪声：技能和工具先提供索引，需要时再加载详细内容，减少 Token 消耗和信息干扰。

第三是压缩历史、保留关键状态：接近配置阈值时，把旧消息压缩成摘要，保留近期原文；目标、待办独立持久化，避免压缩后忘记任务或重复执行。

4. 你们Agent项目采用什么方案支持中断恢复？

结论： 支持中断恢复的核心是LangGraph 的checkpoint持久化机制，Agent 执行完一个图节点之后更新ThreadState，通过Checkpointer保存ThreadState

和图执行信息；使用的时候根据thread_id查询历史ThreadState，合并本轮用户输入；

重点： 中断恢复分为两种情况，一种是等待用户补充信息：当用户提供信息不足的时候，Agent会调用澄清Tool，把问题和结构化选项写入消息，然后保存检查点结束当前执行；当用户回复后，读取到之前保存的会话状态进行下一轮处理。

一种是用户主动停止：前端调用取消接口，后端接收之后，设置停止标记并且调用task.cancel()向正在运行的任务执行取消；Agent捕获取消之后保存检查点；当用户重新发起任务之后，同样读取之前的会话状态进行下一轮处理。

5. 如何评测一个 Agent 的效果？

结论：我们的 Agent 项目接入 Langfuse，通过离线数据集实验和线上持续评估两种方式来进行评测。

离线回归评测： 收集生产中的典型问题，建立包含预期答案的数据集。每次修改提示词、模型或 Agent 逻辑后，通过实验脚本调用真实 Agent 运行测试集，再由评估器评分，并与基线版本比较，检查是否出现能力退步。

线上持续评估： 针对生产真实的Agent执行Trace，按照预先制定的评估规则进行在线评分。然后我们定期分析低分案例，并检查评估是否正常执行，持续发现和定位线上问题。将线上发现的问题持续补充到离线测试集中，形成“发现问题—修复—回归验证”的循环。

6. 线上 Agent 延迟明显升高时，如何通过 Trace 定位和优化？

结论：我们项目接入了Langfuse来做Agent的可观测性；当出现延迟明显升高时，先使用 Langfuse 的仪表盘，观察 P95 延迟确认异常时间和影响范围；然后使用Trace筛选功能，找出同类的正常请求和慢请求，然后打开TimeLine，对比观察各个节点的调用耗时。 对于异常耗时节点进行专项优化之后，使用数据集和评测功能对比优化前后效果，确认没有问题之后再进行上线。

7. Multi-Agent 系统如何处理子 Agent 超时、失联和并发修改冲突？

结论： 子Agent超时和失联问题：主 Agent 通过 `task` 工具派发任务，后端生成独立的 `execution_id`，用于查询、取消和清理。执行器5秒检查一次状态，并且设置60秒的超时时间；当发现断联之后，会请求取消，返回明确的错误，并且安排清理。

并发修改冲突，我们项目是从从任务拆分上避免的。项目当前的调度原则就是：只有输出独立、可变状态不重叠的任务才并行；存在依赖或者

可能修改同一文件时，就串行执行。并行执行时，子Agent写入各自的产物文件，最后由主 Agent 校验和汇总。

8. Agent 的「任务幻觉」是什么？如何避免没有执行工具却声称任务已经完成？

结论：Agent 的「任务幻觉」是指它把计划、尝试或者一段看起来成功的工具输出，当成了真实完成，并向用户作出与外部状态不一致的声明。

我们项目在工具执行结束后，在消息元数据里写入回执，记录 `tool_call_id`、工具名、成功或失败状态；要求元数据中的状态必须为成功，

Agent才能视为工具调用成功。

9. 大模型或 Agent 连接数据库时，如何防止越权、敏感数据泄漏和查询幻觉？

结论： 防止越权，我们不直接使用大模型生成的SQL语句直接执行，而是在执行前，强制在where条件中补齐数据隔离信息，如商户用户在where语句中强制拼上上下文中的商户号，兜底保证了只能查询自己的数据，防止越权；

敏感数据泄露方面，我们基于配置，制定了哪些表的哪些行数据是敏感数据，如订单表的银行卡字段，在工具执行查询后，兜底检查此次是否查询了敏感列，再根据指定的规则进行数据脱敏。

查询幻觉方面，我们分为SQL报错和查询结果不准。当查询报错的时候，根据报错信息，制定重试机制；查询结果不准方面，我们使用了反思机制，引入了校验子Agent，对问题、查询SQL，查询结果进行一次校验，如果有问题，就提出问题，打回给大模型重新查询。

10. 工具很多时，Agent 如何减少 Token 并避免选错工具？

结论：我们维护了统一的工具注册表，给工具补齐命名空间、标签、适用场景；每次请求前，先根据用户权限和当前任务，对工具进行一层硬过滤，把无权限或者不使用的工具直接隐藏。

重点：我们项目中，另外有一套可配置的延迟加载机制，开启后，只在提示词里暴露名称，完整 Schema 暂时不绑定给模型。Agent 根据工具名称调用 `tool_search`，来加载它们的完整定义，让它们进入可调用集合。优点是节省了大量工具的描述和参数定义的token，缺点是需要进行多一次tool_search的工具调用。因此我们这套机制是做成可配置的，默认并不启用，当工具很少的时候反而会起反效果。

11. 工具调用格式非法、参数错误、超时或失败时，Agent 如何容错？

结论：我们项目使用ToolCallMiddleware中间件，保证消息协议的完整，并且给模型可操作的反馈。

当捕获到工具调用错误时，补上tool_call_id的错误结果，转成status="error"的ToolMessage，

详细描述错误性质，让模型读取到之后可以修正后重新调用。

12. 使用 LangChain 构建 Agent 的核心步骤是什么？

结论： 第一步：选择并配置模型。

第二步：定义并注册工具。

第三步：设计系统提示词。

第四步：指定上下文格式。

第五步：配置中间件；

第六步：使用createAgent组装Agent。

13.你的Agent项目是如何做并行工具调用的？

结论：主 Agent 接收到需求后，如果几个操作互不依赖，就在同一条模型响应中返回多个 `tool_calls`；我们通过 `create_agent` 构建执行图，Langraph的工具执行机制会并行分发这些工具的调用，执行完成后，将结果封装为ToolMessage。

14. 如何写好 Prompt？你们项目 Prompt 工程实践经验？

结论：写好 Prompt 核心就是做好五件事：给模型设定角色、说清楚任务、交代背景上下文、约定输出格式、提供示例。

在系统提示词的设计中，我们的重点是模块化，每个模块使用**XML** 标签包裹内容，使用提示词模板统一组装各模块内容。

项目自主问答问题

请你介绍下支付智能助手项目，遇到过什么重难点？

支付智能助手叫 NexAI，内嵌在 NEXGO HUB 支付系统中，主要服务海外代理商和商户。用户可以通过自然语言咨询商户进件、终端配置等业务问题，也可以让助手调用页面操作工具，辅助完成具体业务操作，降低系统使用门槛和人工支持成本。

我主要负责三个方面：一是基于 LangChain 和 LangGraph 实现主、子 Agent 协作和任务编排；二是接入 RAGFlow 支付业务知识库；三是把业务页面操作封装成 Agent 工具，连接问答和操作执行。

这个项目的难点主要集中在以下三个方面。

一个难点是多**Agent**协作机制的设计。

在 NexAI 中，多Agent使用的是监督者模式，将子agent封装成一个task tool供leaderAgent调用，task tool根据配置组装子 Agent 的模型、工具和技能。这里的难点主要是任务拆分、上下文隔离、并发资源管理。

首先是任务拆分。主 Agent 根据任务要求和子 Agent 的能力描述判断是否委派。将相互独立、没有重叠写入的任务并行执行，有可能存在写入冲突的则串行执行。为了防止写入冲突，还让子Agent将文件修改写入各自的工作区，返回报告和产物位置，主Agent统一核查之后合并修改。

第二是上下文隔离。每个子 Agent使用独立状态，默认只接收本次委派任务，避免把主Agent的消息历史放进上下文。同时，用户身份、权限和工作区信息通过运行上下文传递，保证子 Agent仍在正确的权限边界内执行。

第三是并发控制。系统限制一次任务的子Agent调度次数；底层还有进程共享的执行容量，满载时排队或拒绝，防止子agent无限创建耗光服务器资源。

一个难点是：在上下文压缩时，还要保住任务状态

长任务会不断产生搜索结果、文件内容和工具输出，上下文很快膨胀。但直接进行压缩，又可能让 Agent 忘记用户要求、重复委派任务或重新读取技能。

NexAI把关键信息拆成不同状态，包含：当前保留的消息，被压缩的历史摘要，当前的目标和待办，子Agent委派记录，已读取的技能等。

压缩中间件，判断达到配置阈值触发压缩后，会提取状态保存；然后保留近期消息，将历史消息和上一版摘要一起交给大模型进行压缩，生成更新后的摘要。

真正调用模型的时候，持久化上下文包装中间件，会读取最新摘要，当前目标和待办，子gent委派记录等，注入本次请求，这样，模型即使看不到之前几十轮的消息过程，也能知道摘要，当前目标和待办等信息，从而完成最新任务。

最后，我们必须承认摘要会产生信息损失，我们的关注点是尽最大可能的保证Agent在上下文压缩后，能够可靠的接着完成任务。

一个难点是操作页面的系统工具过多，**token**消耗大，上下文噪声大的问题。

我们把页面能力封装成具有明确输入参数和适用条件的业务工具，由于我们系统有一百多个页面，一开始我们根据页面功能划分，拆分出了几百个业务工具，很快发现出现工具过多，上下文噪声大的问题，我们首先治理了工具粒度，合并的标准是操作的业务含义和执行流程相同，只是目标对象或参数不同，就考虑合并，比如页面跳转都统一为一个工具，通过参数确定目标，这样子我们的工具精简了三分之二，但是还有一百多个工具。

之后我们使用业务域路由，将商户域、终端域、交易域、账务域、通用域等各自设置一个专用子agent，主agent只负责理解用户意图和分派任务，每个子agent携带各自的工具定义完成具体执行。这样子每个Agent仅需要分配各自业务需要的工具，大大的降低了Agent的工具上下文。

请你介绍下**HUB**支付系统项目，遇到过什么重难点？

NEXGO HUB 是面向海外代理商和商户的支付系统，我主要负责资金处理和数据服务这两部分。

比较有代表性的难点有两个。

第一个是随着交易订单数据增长到数亿，原有**MySQL** 报表方案出现查询延迟，需要重新设计数据处理和查询链路。

交易报表需要按不同业务维度筛选和汇总。数据量大以后，如果每次查询都直接在交易明细上完成复杂计算，响应时间就会变长。

于是我自学大数据技术栈，基于 Flink、Doris 和 Airflow 搭建了大数据报表方案：通过 Flink CDC 同步业务数据，使用 Doris 承接分析查询，通过 Airflow 编排离线计算任务；

同时采用数据分层的思路，将数据划分成原始数据层ODS、明细数据层DWD、汇总数据层DWS、应用数据层ADS和维度数据层DIM。各层数据职责清晰。

最终，报表查询从数秒优化到毫秒级，并能够支持数十亿级数据的报表查询。

第二个是随着接入渠道增多，峰值**QPS**达到**200**多，压测情况下，当前系统仅支持最大可持续**300**的**QPS**，为了防止出现了消息堆积，记账延迟等情况，我们迫切对记账逻辑进行优化。

综合成本考虑，我们没有直接在服务器层面上加上内存和CPU资源，也没有增加实例数量，而是先从业务代码出发进行优化。

首先我使用阿里开源的arthas定位完整记账业务的Trace，拆解记账业务，统计各环节耗时，然后从以下几方面逐步优化。

一是提高并行度。梳理业务依赖，用受控线程池并行处理互不依赖的查询和计算，如分账和分润的计算逻辑将串行处理改成并行处理，减少串行等待；

二是优化**SQL**和索引。 统一梳理记账业务所有涉及的SQL语句，筛选出高频查询条件字段，进行索引优化。

三是使用缓存。将商户计费套餐、分账规则等高频使用的信息redis缓存中，避免每次记账都从MYSQL中读取。

四是缩小锁范围。仅在涉及记账账户和余额扣减等核心操作加分布式锁，缩短锁持有时间，降低锁竞争。

然后我们通过标准配置持续压测，对比吞吐量，单笔记账时间；

并且使用不同的服务器配置和实例数量，进行多组压测验证，对比配置对吞吐量的产生的影响，当业务持续发展且代码层面没有优化空间的时候，可以直接根据准备好的数据进行合适的硬件扩容。

请你介绍下**Agent**开发学习平台项目，遇到过什么重难点？

Agent 开发学习平台是我独立开发的个人开源项目，面向想学习 AI 和 Agent 开发的开发者，提供学习路线资料、面试题库、AI 智能刷题和刷题记录分析等功能。

这个项目中，我觉得最有挑战性是难点是作为一个后端+Agent开发，配合AI，独立完成了前后端+Agent全栈项目的开发，以及线上部署，且项目收获了一定程度的认可。

中间仔细打磨了正反馈机制的AI智能刷题功能，建立勋章和等级制度。尽可能给用户带来游戏性的刷题体验，让用户更有驱动力的刷题。

请你介绍下薪福通项目，遇到过什么重难点？

薪福通是招商银行面向企业客户的一站式数字化管理 SaaS 平台，主要覆盖组织管理、人事、财务费控和协同办公等场景。

我在这个项目中主要负责三个方面：一是企业认证、组织机构、权限管理等基础能力；二是员工入转调离、历史任职等人事功能；三是对接钉钉、飞书、企业微信，实现企业组织和人员数据同步。

比较有代表性的难点有两个。

第一个是多平台组织和人员同步，难点在于需要兼容某些大企业的数据量，而且要保证组织层级和人员归属正确。

不同平台的数据结构和 ID 体系不一样，组织之间又存在父子关系，不能简单地拉取数据后直接批量入库。

例如，父部门还没有创建，子部门就无法正确关联；部门没有同步完成，人员归属也可能出现问题。另外，同一家企业重复触发同步，还可能造成任务互相干扰。

针对这些问题，我通过 ID 映射维护外部平台与内部系统的数据关联，使用广度优先遍历算法，按照组织树的父子层级顺序处理部门，再同步人员，并使用企业级同步锁避免同一企业的同步任务并发冲突。

同时，对不同渠道的差异分别进行处理。

最终，这套方案支撑了上万家企业的组织和人员数据同步，大企业初始化配置时间从平均几天缩短到了半小时。

第二个是亿级历史任职数据的查询性能治理。

当时历史任职记录已经达到约 1.28 亿条，原有查询方案的平均响应时间在3秒左右。

经过技术评审，我们决定启用Elasticsearch来承接历史任职记录的数据。

迁移的时候有一个难点：业务持续运行、数据不断变化，还要保证迁移期间和迁移后的查询结果完整，并且当时由于项目背景，不允许使用Flink CDC和Canal等数据同步组件。

定时任务迁移存量数据、增量数据双写和双删，以及双查结果合并等方案；处理历史数据和持续变更数据的衔接问题。

优化后，平均查询响应时间从 3 秒降低到了 150 毫秒左右。

网上面经